

Unity URP 핵심 강의

Universal Render Pipeline의 구조, 설정, 렌더러, 카메라, 조명, 디버깅



문서 기준

프로젝트 기준: Unity 6000.3.9f1, Universal Render Pipeline 17.3.0

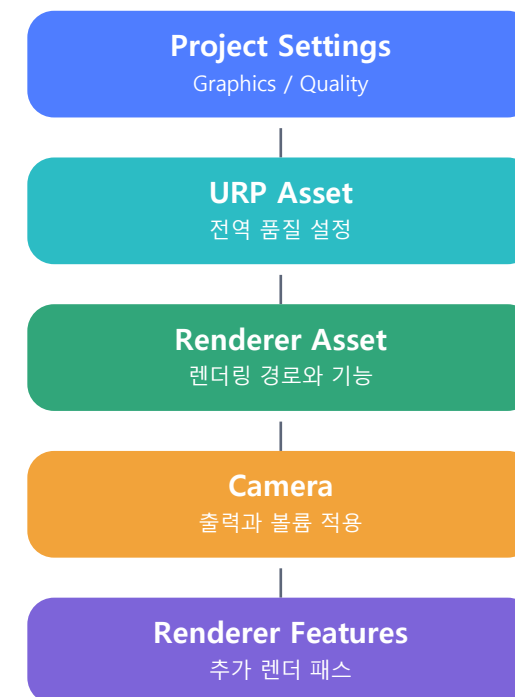
목표: URP를 '옵션 모음'이 아니라 렌더링 구조로 이해하기

범위: URP Asset, Renderer, Rendering Path, Renderer Feature, Render Graph, Camera, Volume, Lighting

출처: Unity 공식 문서 중심, 모든 외부 문서는 2026-07-20 기준 확인

0강. URP를 배우는 이유

- URP는 Unity의 Scriptable Render Pipeline 중 하나로, 여러 플랫폼을 대상으로 확장 가능한 렌더링을 제공한다.
- 현재 프로젝트는 Unity 6000.3.9f1과 Universal Render Pipeline 17.3.0을 사용한다.
- URP를 잘 다룬다는 것은 단순히 체크박스를 켜고 끄는 것이 아니라, Asset, Renderer, Camera, Volume, Shader, Lighting의 관계를 이해하는 것이다.
- 렌더 최적화 포트폴리오에서는 URP 설정을 바꾼 이유와 측정 결과를 함께 설명할 수 있어야 한다.



1강. URP는 SRP 위에 있는 렌더 파이프라인이다

Built-in

Unity의 전통적인 렌더 파이프라인. 기존 자료가 많지만 SRP 기반 커스터마이징 구조와 다르다.

URP

범용 플랫폼을 목표로 하는 SRP 기반 렌더 파이프라인. PC, 모바일, 콘솔 등 다양한 환경에 맞춘다.

U3

HDRP

고품질 하이엔드 그래픽을 목표로 하는 SRP 기반 렌더 파이프라인. URP와 동시에 사용할 수 없다.

- SRP Batcher는 SRP에서 CPU 렌더 준비 비용을 줄이는 주요 최적화 경로 중 하나다.
- URP 프로젝트를 다룰 때는 Project Settings, URP Asset, Renderer Asset, Camera 설정을 함께 읽어야 한다.

2강. URP 프로젝트에서 먼저 봐야 할 파일

Packages/manifest.json

URP 패키지 버전 확인. 현재 프로젝트는 com.unity.render-pipelines.universal 17.3.0.

ProjectSettings/GraphicsSettings

프로젝트 기본 Scriptable Render Pipeline 설정 확인.

ProjectSettings/QualitySettings

품질 레벨별 Render Pipeline Asset 연결 확인.

Assets/Settings/*_RPAsset

URP Asset. Render Scale, HDR, MSAA, Shadows, Lighting 같은 전역 품질 설정.

Assets/Settings/*_Renderer

Renderer Asset. Rendering Path, Renderer Features, Layer Mask, Intermediate Texture 등.

3강. URP Asset은 전역 품질 설정이다

- URP Asset은 RenderPipelineAsset을 상속하는 ScriptableObject이며, URP의 그래픽 기능과 품질 설정을 제어한다.
- Graphics Settings에 URP Asset을 할당하면 Unity가 Built-in Render Pipeline 대신 URP를 사용한다.
- 하나의 프로젝트에 여러 URP Asset을 둘 수 있고, 예를 들어 Shadows On/Off 같은 설정 차이를 Asset 단위로 비교할 수 있다.
- 현재 프로젝트에는 PC_RPAsset과 Mobile_RPAsset이 있으므로 PC/Mobile 품질 비교 실습에 적합하다.

PC_RPAsset

Mobile_RPAsset

Quality Level

Graphics

4강. URP Asset에서 자주 만지는 옵션

Render Scale

렌더링 해상도 비율. 낮추면 GPU 비용이 줄 수 있지만 선명도 손실이 생긴다.

U4

HDR

고명도 표현과 Bloom에 유리하지만 렌더 타킷 비용과 플랫폼 비용을 고려한다.

U4

MSAA

기하 경계의 계단 현상을 줄인다. 샘플 수가 커질수록 비용이 증가한다.

U4

Depth/Opaque Texture

특정 셰이더/효과에 필요할 수 있지만 텍스처 생성과 복사 비용을 만든다.

U4

Lighting/Shadows

Main/Additional Light, shadow distance, cascade, resolution은 성능 영향이 크다.

U4

SRP Batcher

호환 shader/material에서 CPU 렌더 준비 비용을 줄이는 경로다.

U4

5강. Renderer Asset은 '어떻게 그릴지'를 정한다

- Universal Renderer Asset은 URP Asset이 사용할 렌더러 설정을 담는다.
- Renderer Asset에서는 Rendering Path, Opaque/Transparent Layer Mask, Intermediate Texture, Renderer Features를 설정한다.
- URP Asset 하나가 여러 Renderer를 가질 수 있고, Camera마다 사용할 Renderer를 지정할 수 있다.
- 렌더 최적화 실습에서는 PC_Renderer와 Mobile_Renderer의 Renderer Feature 차이를 먼저 확인한다.



생각법

URP Asset은 전역 품질,
Renderer Asset은 프레임
안의 렌더링 구성으로 나누어
기억하면 좋다.

6강. Forward, Forward+, Deferred

Forward

가벼운 기본값
MSAA 가능
조명 수 제한

Forward+

많은 조명 대응
per-camera 제한
모바일도 검토

Deferred

불투명 조명에 강함
G-buffer 비용
MSAA 제약

- Forward는 기본 경로이며 조명이 많지 않거나 모바일/저사양 플랫폼에서 우선 고려된다.
- Forward+는 많은 조명이 필요한 경우, Deferred가 목표 플랫폼에서 느릴 때 검토할 수 있다.
- Deferred는 많은 불투명 조명 처리에 강점이 있지만 G-buffer 렌더 타깃과 플랫폼 제약을 함께 고려해야 한다.
- 처음 실습에서는 Rendering Path 자체보다 조명 수와 그림자 설정이 프레임에 주는 영향을 먼저 본다.

7강. Forward Rendering을 먼저 이해한다

- Forward Rendering은 많은 URP 프로젝트에서 기본 출발점이 된다.
- 불투명 오브젝트와 투명 오브젝트를 비교적 직관적인 방식으로 그리며, MSAA 사용이 가능한 경로다.
- 추가 조명 수가 많아질수록 per-object/per-camera 조명 제한과 비용을 확인해야 한다.
- 캐릭터 라이브 씬처럼 투명 머리카락, 의상, UI, 이펙트가 섞이면 Forward에서 투명 오브젝트 비용을 따로 측정해야 한다.

첫 실험

Forward + Main Light +
Additional Lights 0/4/8
조건으로 FPS, GPU ms,
SetPass를 비교한다.

U6

8강. Deferred는 조명 처리와 G-buffer를 같이 본다

- Deferred는 불투명 오브젝트의 조명 처리에 강점이 있지만 G-buffer 렌더 타깃을 사용하므로 메모리/대역폭 비용이 생긴다.
- URP 문서는 모바일에서 Deferred가 추가 렌더 패스 때문에 비용이 커질 수 있음을 설명한다.
- 투명 오브젝트는 Deferred에서도 Forward 방식으로 렌더링될 수 있으므로, 투명 캐릭터 파츠가 많은 씬은 따로 확인해야 한다.
- Deferred를 선택하기 전에 목표 플랫폼, 조명 수, MSAA 필요 여부, Render Layers 사용 여부를 같이 점검한다.

판단 기준

조명 수가 많아도
모바일/투명/메모리 조건
때문에 무조건 Deferred가
답은 아니다.

U6

9강. Renderer Feature는 렌더 패스를 추가하는 장치다

Render Objects

특정 레이어를 다른 시점에 그리거나 material/depth/stencil override를 적용한다.

U7

SSAO

화면 공간에서 접힌 부분과 가까운 표면을 어둡게 만들어 입체감을 준다.

U7

Screen Space Shadows

주 방향광 그림자를 화면 공간에서 처리하는 기능. 추가 렌더 타깃 비용이 있다.

U7

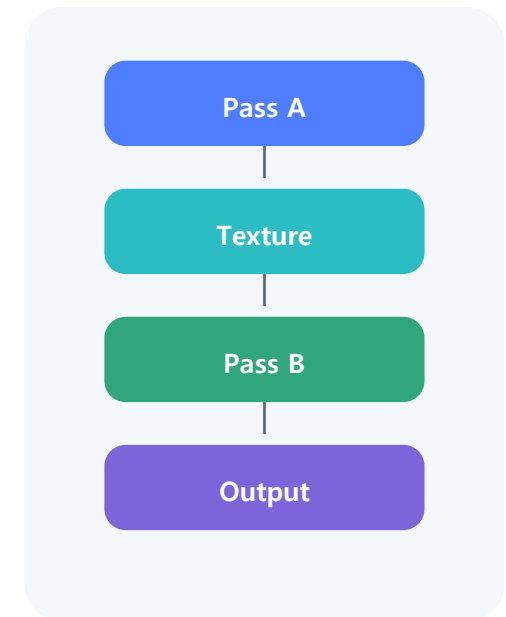
Full Screen Pass

전체 화면 효과와 커스텀 포스트 프로세싱을 비교적 적은 코드로 추가한다.

U7

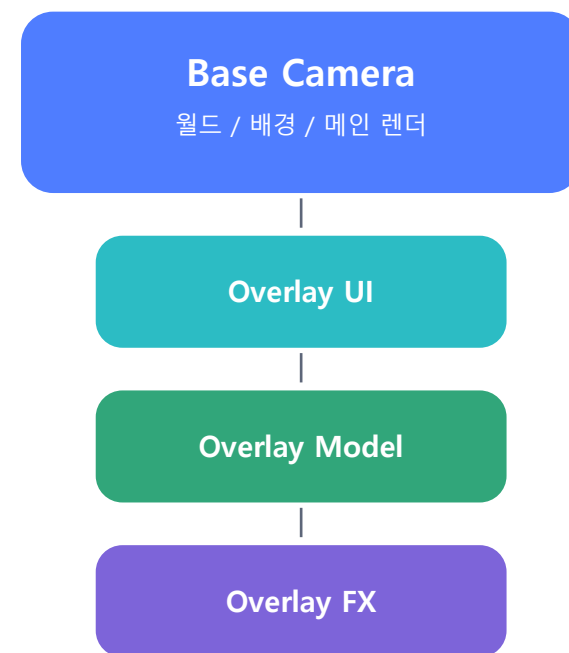
10강. URP 17의 큰 변화: Render Graph

- Unity 6의 URP 17에는 Render Graph 시스템이 도입되었다.
- Render Graph는 스크립터블 렌더 패스를 만들기 위한 API 세트이며, 프레임에서 실제로 사용하는 리소스 중심으로 렌더링을 구성한다.
- Unity 문서는 Render Graph가 URP의 메모리 사용을 줄이고 리소스 관리를 더 효율적으로 만든다고 설명한다.
- 초급 단계에서는 Render Graph 코드를 바로 작성하기보다, Frame Debugger와 Rendering Debugger에서 패스 흐름을 읽는 것부터 시작한다.



11강. URP 카메라는 Base와 Overlay로 나뉜다

- URP에는 Base Camera와 Overlay Camera라는 두 가지 카메라 렌더 타입이 있다.
- Base Camera는 화면이나 Render Texture 같은 렌더 타겟에 렌더링하는 범용 카메라다.
- Overlay Camera는 다른 카메라 출력 위에 렌더링하며, Base Camera의 camera stack에 포함되어야 한다.
- 여러 카메라는 아무것도 렌더링하지 않아도 렌더링 루프를 거칠 수 있으므로 성능 비용을 고려한다.



12강. Camera Stack은 레이어와 컬링 마스크가 핵심이다

- Camera Stack은 여러 카메라 출력을 하나의 결합된 출력으로 쌓아 올린다.
- Overlay Camera로 렌더링하려는 오브젝트를 별도 레이어에 배치하고, 각 카메라의 Culling Mask를 맞춘다.
- URP 문서는 불필요한 레이어를 제거하면 렌더링 속도가 더 빨라질 수 있다고 설명한다.
- 라이브/방송형 씬에서는 월드 카메라, 캐릭터 클로즈업, UI/효과용 오버레이 카메라 구조를 실험해볼 수 있다.

실습 아이디어

Base Camera는 배경과 캐릭터를, Overlay Camera는 UI 또는 특정 3D 소품만 그리게 하여 Culling Mask 차이를 측정한다.

13강. Volume은 URP 포스트 프로세싱의 공간 규칙이다

- URP에서는 Volume을 사용해 포스트 프로세싱 효과를 씬 전체 또는 특정 공간에 적용한다.
- Volume Profile에는 Bloom, Color Adjustments, Depth of Field 같은 override가 들어간다.
- Global Volume은 씬 전체에 영향을 주고, Local Volume은 collider/범위와 blending 규칙을 통해 영향을 준다.
- Volume은 시각 품질을 빠르게 바꾸는 좋은 수단이지만, Bloom/Depth of Field/SSAO 같은 효과는 GPU 비용을 반드시 측정한다.



14강. Post-processing은 화면 전체를 다시 만든다

- URP는 다양한 미리 빌드된 포스트 프로세싱 효과를 제공하고, 커스텀 포스트 프로세싱도 만들 수 있다.
- 포스트 프로세싱은 렌더된 화면 결과를 기반으로 하므로 해상도, Render Scale, HDR, 렌더 타겟 비용과 함께 봐야 한다.
- Full Screen Pass Renderer Feature는 전체 화면 효과를 추가하는 실용적인 확장 지점이다.
- 처음 실험은 Bloom, Vignette, Color Adjustments처럼 차이가 보이는 효과부터 On/Off 비교한다.

포트폴리오 포인트

효과 자체보다 '어떤 효과가 몇 ms를 쓰는지'를 보여주는 쪽이 개발자 포트폴리오에 더 강하다.

15강. URP Shader는 품질과 비용의 언어다

Lit

물리 기반 조명 표현. 품질은 좋지만 계산 비용을 고려한다.

U13

Simple Lit

단순 조명 모델. 모바일/저사양에서 검토하기 좋다.

U13

Baked Lit

실시간 조명 없이 라이트맵/프로브 기반 조명을 사용한다.

U13

Unlit

실시간/베이킹 조명의 영향을 받지 않는다. UI, 이펙트, 스타일 표현에 유용하다.

U13

16강. URP Lighting은 Main Light와 Additional Lights부터 본다

- URP Asset은 Main Light와 Additional Lights 관련 설정을 제공한다.
- Forward 경로에서는 per-object real-time light 수 제한을 고려해야 한다.
- 조명 수가 늘수록 렌더링 경로, 그림자 설정, 투명 오브젝트, 카메라 수를 함께 봐야 한다.
- 처음 실습에서는 Directional Light 1개와 Point/Spot Light 여러 개를 배치하고 Additional Light Shadow On/Off를 비교한다.

지원 포트폴리오 문장

URP의 Main/Additional Light 조건을 분리하고, 조명 수와 그림자 옵션이 GPU frame time에 주는 영향을 측정했습니다.

17강. Shadow는 URP 최적화의 첫 번째 큰 산이다

- Unity 문서는 shadow caster 수, shadow receiver 수, shadow-casting light 수, cascade count, shadow resolution, soft shadow 설정이 그림자 렌더링 시간에 영향을 준다고 설명한다.
- Point Light 그림자는 여섯 방향의 shadow map이 필요하므로 비용을 특히 조심한다.
- 그림자 문제는 Shadow Bias 조정으로 shadow acne, peter panning 같은 결함을 줄일 수 있다.
- 성능 실험은 Shadow Distance, Cascade Count, Resolution, Soft Shadow를 한 번에 하나씩 바꾸며 비교한다.

기억할 말

그림자는 품질이 잘 보이고
비용도 잘 보인다. 그래서
첫 최적화 사례로 좋다.

18강. Depth Texture와 Opaque Texture는 필요한 효과가 있을 때 켜다

- Depth Texture는 깊이 기반 효과, 셰이더, 후처리에서 필요할 수 있다.
- Opaque Texture는 불투명 렌더 결과를 샘플링해야 하는 효과에 필요할 수 있다.
- 이 옵션들은 기능 구현에는 편하지만 추가 텍스처 생성/복사 비용이 생길 수 있으므로 항상 측정한다.
- Renderer의 Intermediate Texture 설정도 Renderer Feature 호환성과 성능에 영향을 줄 수 있다.

실습 규칙

효과가 필요해서 켜 옵션인지, 템플릿에서 켜져 있어서 남아 있는 옵션인지 구분한다.

19강. URP를 디버깅하는 기본 도구

Rendering Debugger

조명, 렌더링, 머티리얼 속성을 시각화해 렌더링 이슈와 설정 최적화 지점을 찾는다.

U16

Frame Debugger

한 프레임을 캡처하고, 해당 프레임을 구성하는 draw call과 render event를 순서대로 확인한다.

U17

- URP를 공부할 때는 결과 화면만 보지 말고, Frame Debugger로 pass 순서를 읽는 습관을 들인다.
- Rendering Debugger는 Development Build에서도 사용할 수 있으므로 실제 기기 확인에 도움이 된다.

20강. Built-in 프로젝트를 URP로 옮길 때 주의할 점

- Render Pipeline Converter는 Built-in Render Pipeline용 에셋을 URP 호환 에셋으로 전환한다.
- Unity 문서는 전환 과정이 되돌릴 수 없는 변경을 적용하므로 전환 전 백업하라고 명시한다.
- Material Upgrade는 Unity가 제공하는 pre-built material에는 도움을 주지만 custom shader는 수동 업그레이드가 필요할 수 있다.
- 기존 프로젝트를 URP로 옮길 때는 렌더링 설정, 머티리얼, 포스트 프로세싱, 애니메이션에서 머티리얼 속성을 건드리는 부분을 점검한다.

실무 감각

URP 전환은 버튼 한 번으로 끝나는 작업이 아니라, 렌더 결과와 성능을 다시 검증하는 마이그레이션 작업이다.

21강. 이 프로젝트에서 URP를 공부하는 순서

- 1 PC_RPAsset / Mobile_RPAsset 값 비교
- 2 PC_Renderer / Mobile_Renderer의 Renderer Features 비교
- 3 SampleScene의 Volume Profile 확인
- 4 Benchmark_Field 씬에서 Render Scale, HDR, MSAA 실험
- 5 Light/Shadow 구역에서 Additional Light와 Cascade 실험
- 6 Frame Debugger로 실제 pass와 draw event 확인

22강. URP 설정을 바꿀 때 체크리스트

- ☐ 이 옵션은 URP Asset 옵션인가, Renderer Asset 옵션인가, Camera 옵션인가?
- ☐ 이 옵션이 필요한 기능은 무엇인가?
- ☐ 품질 손실은 무엇이고, 측정할 지표는 무엇인가?
- ☐ Editor 수치와 Build 수치를 구분했는가?
- ☐ Frame Debugger에서 실제 pass가 줄었거나 바뀌었는가?
- ☐ PC_RPAsset과 Mobile_RPAsset의 차이를 문서화했는가?
- ☐ 한 번에 하나의 변수만 바꾸고 비교했는가?

23강. URP 최소 용어 사전

URP Asset

URP의 전역 렌더링 기능과 품질 설정을 담는 Asset.

Renderer Asset

렌더링 경로, Layer Mask, Renderer Feature 등을 담는 Asset.

Rendering Path

Forward, Forward+, Deferred처럼 조명과 shading을 계산하는 방식.

Renderer Feature

URP Renderer에 추가 render pass를 넣고 동작을 설정하는 Asset.

Render Graph

URP 17에서 도입된 렌더 패스와 리소스 관리 시스템.

Base Camera

렌더 타겟에 직접 렌더링하는 URP 기본 카메라.

Overlay Camera

Base Camera 출력 위에 렌더링되는 카메라.

Volume

포스트 프로세싱과 override를 공간 또는 전역으로 적용하는 시스템.

24강. 다음 실습으로 이어지는 과제

- PC_RPAsset과 Mobile_RPAsset의 주요 설정을 표로 정리한다.
- PC_Renderer와 Mobile_Renderer의 Renderer Features를 비교한다.
- Benchmark_Field 씬을 만들고 Forward 경로에서 Light/Shadow 비용을 측정한다.
- Frame Debugger로 Shadow, Depth, Opaque, Transparent, Post pass 흐름을 캡처한다.
- Render Scale 1.0 / 0.8 / 0.67 조건에서 GPU frame time과 화면 품질을 비교한다.

이번 자료의 핵심

URP는 '최적화 옵션 목록'이 아니라 Asset, Renderer, Camera, Volume, Shader, Lighting이 연결된 렌더링 구조다.

참고문헌 및 도식 출처

확인일: 2026-07-20. 본 PDF의 도식은 외부 이미지를 복제하지 않고 강의용으로 직접 제작한 벡터 도식이며, 각 페이지의 근거 태그는 아래 문헌을 의미한다.

[U1] Project Unity version

local project file - ProjectSettings/ProjectVersion.txt

[U2] Project package manifest

local project file - Packages/manifest.json

[U3] URP features | Universal RP 17.0.0

Unity Technologies - <https://docs.unity.cn/Packages/com.unity.render-pipelines.universal%4017.0/manual/urp-feature-list.html>

[U4] Universal Render Pipeline Asset | Universal RP 17.0.0

Unity Technologies - <https://docs.unity.cn/Packages/com.unity.render-pipelines.universal%4017.0/manual/universalsrp-asset.html>

[U5] Universal Renderer | Universal RP 17.0.0

Unity Technologies - <https://docs.unity.cn/Packages/com.unity.render-pipelines.universal%4017.0/manual/urp-universal-renderer.html>

[U6] Choose a rendering path in URP | Unity 6

Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/urp/rendering-paths-comparison.html>

[U7] URP Renderer Feature | Universal RP

Unity Technologies - <https://docs.unity.cn/Packages/com.unity.render-pipelines.universal%4016.0/manual/urp-renderer-feature.html>

[U8] Render graph system in URP | Unity 6

Unity Technologies - <https://docs.unity3d.com/kr/current/Manual/urp/render-graph.html>

[U9] Multiple cameras in URP | Unity 6

Unity Technologies - <https://docs.unity3d.com/kr/6000.0/Manual/urp/cameras-multiple.html>

[U10] Camera render types in URP | Unity 6

Unity Technologies - <https://docs.unity3d.com/kr/current/Manual/urp/camera-types-and-render-type-introduction.html>

참고문헌 계속

확인일: 2026-07-20. 본 PDF의 도식은 외부 이미지를 복제하지 않고 강의용으로 직접 제작한 벡터 도식이며, 각 페이지의 근거 태그는 아래 문헌을 의미한다.

[U11] Volumes in URP | Unity 6

Unity Technologies - <https://docs.unity3d.com/kr/6000.0/Manual/urp/volumes-landing-page.html>

[U12] Custom post-processing in URP | Unity 6

Unity Technologies - <https://docs.unity3d.com/kr/6000.0/Manual/urp/post-processing/custom-post-processing.html>

[U13] Shading models in URP | Unity 6

Unity Technologies - <https://docs.unity3d.com/jp/current/Manual/urp/shading-model.html>

[U14] Optimize shadow rendering in URP | Unity 6

Unity Technologies - <https://docs.unity3d.com/6000.0/Documentation/Manual/shadows-optimization.html>

[U15] Troubleshooting shadows in URP | Unity 6

Unity Technologies - <https://docs.unity3d.com/kr/6000.0/Manual/urp/shadows-troubleshooting-urp.html>

[U16] Rendering Debugger | Universal RP

Unity Technologies - <https://docs.unity.cn/Packages/com.unity.render-pipelines.universal%4015.0/manual/features/rendering-debugger.html>

[U17] Debug a frame | Unity 6 Frame Debugger

Unity Technologies - <https://docs.unity.cn/6000.0/Documentation/Manual/FrameDebugger-debug.html>

[U18] Render Pipeline Converter | Unity 6

Unity Technologies - <https://docs.unity3d.com/kr/current/Manual/urp/features/rp-converter.html>

[U19] Scriptable Render Pipeline Batcher

Unity Technologies - <https://docs.unity.cn/Manual/SRPBatcher.html>

[U20] What's new in URP 17 (Unity 6)

Unity Technologies - <https://docs.unity.cn/Manual/urp/whats-new/urp-whats-new.html>